Introduction

Welcome back.

In the previous video, we created the foundation of our multimodal RAG application using ASP.NET Core.

We created the MVC project, built the upload form, created the RAG controller, and successfully uploaded document and image files.

In this video, we will take the next important step.

We will extract readable text from uploaded document files.

This is a critical step in any RAG pipeline because before we can chunk text, create embeddings, store vectors, or generate AI responses, we first need to convert the uploaded document into plain text.

So in this part, we will support two document types:

TXT files and PDF files.

Why Text Extraction Matters

Let us quickly understand why document text extraction is important.

A RAG system cannot directly search inside a raw PDF file.

First, the content has to be extracted as plain text.

Once the text is available, we can split it into chunks.

Then we can create embeddings from those chunks.

Then those embeddings can be stored in a vector database.

And later, when a user asks a question, the system can retrieve the most relevant chunks and use them to generate an answer.

So this video is about the first real processing step in the RAG pipeline.

Service-Oriented Explanation

We will not put document extraction logic directly inside the controller.

That would make the controller messy.

Instead, we will create a dedicated service called DocumentTextExtractor.

The controller will only handle the request.

The service will handle the document processing.

This is a cleaner approach and much closer to real-world application design.

Interface Explanation

First, we create an interface called IDocumentTextExtractor.

This interface contains one method: ExtractTextAsync.

The method accepts a file path and returns extracted text.

Using an interface gives us flexibility.

Later, we can add more implementations, support more file types, or test this service separately.

Service Code Explanation

Now we create the DocumentTextExtractor class.

This class implements IDocumentTextExtractor.

Inside the ExtractTextAsync method, we first check whether the file path is valid.

Then we check whether the file actually exists.

After that, we look at the file extension.

If the extension is TXT, we simply read the file using File.ReadAllTextAsync.

If the extension is PDF, we use PdfPig to open the PDF document.

Then we loop through all the pages and extract text from each page.

Finally, we combine the page text into a single string and return it.

Dependency Injection Explanation

Next, we register the service in Program.cs.

This allows ASP.NET Core to create and inject the service automatically.

So when the controller asks for IDocumentTextExtractor, the framework provides DocumentTextExtractor.

This is standard dependency injection in ASP.NET Core.

Controller Update Explanation

Now we update the RAG controller.

Previously, after saving the uploaded document, we only displayed the filename.

Now, after saving the file, we also pass the saved file path to the document extraction service.

The service returns the extracted text.

For now, we put that text into ViewBag and send it back to the view.

This allows us to verify that document extraction is working before moving to the next stage.

Razor View Explanation

Finally, we update the Razor View.

If extracted text is available, we display it inside a Bootstrap card.

We use a pre tag so that line breaks and formatting are preserved.

We also add a maximum height and scrolling because extracted PDF text can be long.

This gives us a simple way to visually confirm that the document has been processed successfully.

Demo Script

Now let us run the application.

I will upload a simple TXT file first.

After submitting the form, we can see that the application displays the uploaded filename and the extracted text.

Now let us try a PDF file.

If the PDF contains selectable text, PdfPig can extract that text and display it here.

This confirms that the second step of our RAG pipeline is working.

Closing

In this video, we added document text extraction to our multimodal RAG application.

We created a clean document extraction service, supported TXT files, supported PDF files using PdfPig, registered the service with dependency injection, updated the controller, and displayed the extracted text in the Razor View.

This gives us the raw text needed for the next stage of the RAG pipeline.

In the next video, we will take this extracted text and split it into smaller chunks.

Chunking is extremely important because large documents cannot be sent directly to an embedding model or chat model efficiently.

So next, we will build a TextChunkingService and prepare our document content for embeddings and vector search.